

SÍLABO 2026-II

Data Science con Python

Universidad del Pacífico — Departamento de Economía

1. Información General

Nombre del curso	Data Science con Python
Semestre	2026-II (10 de agosto al 21 de noviembre de 2026)
Profesor del curso	Alexander Quispe Rojas
Email	aw.quispero@up.edu.pe, alexander.quispe@pucp.edu.pe
Jefes de prácticas	Paul Melo Ramos, Jean Pool Nieto
Email de prácticas	PA.MeloR@up.edu.pe, j.nietoc@up.edu.pe
Horas Teoría	2 Horas, martes 7:30–9:20 am
Horas Práctica	2 Horas, miércoles 7:30–9:20 am
Asesoría	https://calendly.com/alexander-quispe
Curso web	https://d2cml-ai.github.io/Data-Science-Python/

2. Sumilla

Este curso ofrece una visión integral del ecosistema de Python aplicado a la ciencia de datos, recorriendo la cadena completa de valor de un producto moderno de datos: desde la gestión de proyectos con Git y GitHub y los fundamentos del lenguaje, hasta la recolección de datos por *web scraping* y APIs, el análisis geoespacial, y la construcción de aplicaciones con modelos de lenguaje de gran tamaño. El curso incorpora las herramientas que definen la práctica profesional actual: **agentes de código** como entorno de desarrollo, **servidores MCP** para conectar agentes con servicios y fuentes de datos, **RAG** y *fine-tuning* de modelos abiertos, **document AI** y reconocimiento de voz, y el **despliegue** de productos accesibles al público. El semestre culmina con un proyecto individual: cada estudiante construye y despliega una **startup funcional** con todo el flujo de datos operando de extremo a extremo.

3. Presentación

El propósito de este curso es capacitar a las y los estudiantes en el uso de Python como herramienta versátil para el desarrollo de proyectos y la resolución de problemas complejos que involucran datos. Iniciaremos con una descripción de qué significa la ciencia de datos y su flujo de trabajo, y con el control de versiones mediante Git y GitHub, que garantiza colaboración ordenada y trazabilidad. Desde la primera sesión incorporamos los **agentes de código** —Claude Code, Codex— no como curiosidad sino como el entorno de trabajo real de quien programa hoy.

A continuación recorreremos la recolección y manipulación de datos: *web scraping* de sitios dinámicos y consumo de APIs, y luego la conexión de un agente de código con servicios externos mediante **MCP**: GitHub, Gmail y WhatsApp. Sobre esa base trabajaremos visualización, *dashboards* y análisis geoespacial vectorial y raster.

El bloque final está dedicado a la inteligencia artificial aplicada: modelos de lenguaje, salidas estructuradas, recuperación aumentada, *fine-tuning*, digitalización de documentos, transcripción de voz y agentes. El curso cierra con **despliegue y evaluación**: cómo poner un producto en línea y cómo saber que funciona.

La diferencia de este semestre está en el entregable final. No se pide un *notebook* bonito ni un informe: se pide una **startup consolidada**, con su flujo de datos completo funcionando y desplegada en una URL pública, como la versión de software o aplicación que se ofrecería al mercado.

4. Resultados de Aprendizaje

Al finalizar el curso, las y los estudiantes serán capaces de:

- **Aplicar el flujo de trabajo de la ciencia de datos**, usando Git y GitHub para gestionar versiones de forma colaborativa y ordenada, y agentes de código como entorno de desarrollo.
- **Dominar los fundamentos de Python**: sintaxis, funciones, clases y organización modular del código.
- **Extraer datos** de sitios web dinámicos con Selenium y de servicios mediante APIs, y **conectar un agente de código** a servicios externos mediante MCP.
- **Crear visualizaciones y *dashboards*** efectivos con seaborn y Streamlit.
- **Realizar análisis geoespacial** con datos vectoriales y raster, generando mapas estáticos y dinámicos.
- **Construir aplicaciones con LLMs**: diseño de instrucciones, salidas estructuradas, recuperación aumentada y *fine-tuning* de modelos abiertos.
- **Digitalizar documentos y audio** mediante OCR y reconocimiento automático de voz, con extracción estructurada.
- **Orquestar agentes** que ejecutan tareas de varios pasos.
- **Desplegar un producto** en una URL pública y **evaluar** que funciona: pruebas, control de costos y monitoreo.

5. Contenido del Curso

1. Ciencia de datos, Git y GitHub, y agentes de código. Qué es la ciencia de datos y cuál es su flujo de trabajo. Control de versiones: repositorios, ramas, *merges*, resolución de conflictos y *pull requests*. Configuración del entorno de desarrollo con agentes de código (Claude Code, Codex) y buenas prácticas para trabajar con ellos.

2. Fundamentos de Python. Sintaxis básica, tipos de datos y estructuras de control. Funcio-

nes y clases; principios de programación orientada a objetos. Gestión de entornos y dependencias.

3. Web Scraping y APIs. Extracción de información de sitios dinámicos con Selenium: automatización de navegadores, manejo de tiempos de espera y localización de elementos. Consumo de APIs y manejo de formatos de respuesta (JSON, XML), autenticación y límites de tasa. Consideraciones éticas y legales de la recolección automatizada.

4. MCPs en Claude Code: GitHub, Gmail y WhatsApp. El *Model Context Protocol* como forma estándar de conectar un agente de código a servicios externos. Instalación y uso de los MCP de **GitHub** (repositorios, *issues*, *pull requests*), **Gmail** (lectura, búsqueda y redacción de correo) y **WhatsApp** (mensajería). Permisos, credenciales y límites de seguridad al dar acceso a un agente. Como extensión, construcción de un servidor MCP propio que exponga una fuente de datos peruana.

5. Visualización y *dashboards*. Gráficos con matplotlib y seaborn: configuración, estilos y patrones de diseño. Construcción de aplicaciones interactivas con Streamlit y su publicación.

6. Análisis geoespacial: mapas estáticos y dinámicos. Georreferenciación y geocodificación. Manipulación de datos espaciales (*shapefiles*, GeoJSON) con GeoPandas y generación de mapas cropléticos. Mapas interactivos con Folium: capas, controles y animaciones.

7. Datos raster. Manejo de imágenes satelitales y modelos de elevación. Lectura, procesamiento y visualización de rasters, y su combinación con datos vectoriales. Aplicaciones con luces nocturnas y cobertura del suelo.

8. LLMs: instrucciones, salidas estructuradas y RAG. Fundamentos de los modelos de lenguaje. Diseño de instrucciones. **Salidas estructuradas** y llamada a funciones para integrar un modelo dentro de una aplicación. Generación aumentada por recuperación: *embeddings*, bases vectoriales y evaluación de la recuperación.

9. Hugging Face y *fine-tuning*. Uso de modelos preentrenados del ecosistema Hugging Face. Ajuste fino de modelos abiertos sobre datos propios: preparación del conjunto, entrenamiento y evaluación. Cuándo conviene ajustar un modelo y cuándo basta con instrucciones o recuperación.

10. Document AI y voz. Digitalización de documentos con OCR (PaddleOCR) y extracción estructurada de campos mediante modelos de lenguaje. Transcripción de audio con Whisper y comparación de motores de reconocimiento de voz. Casos de uso en expedientes, facturas, actas y grabaciones.

11. Agentes. Orquestación de agentes que ejecutan tareas de varios pasos: crewAI, subagentes y *skills*. Agentes personales autoalojados conectados a aplicaciones de mensajería (OpenClaw). Diseño de herramientas y límites de seguridad.

12. Despliegue y evaluación de productos de datos. Empaquetado con Docker y publicación en Streamlit Cloud, Railway, Render o Hugging Face Spaces. Variables de entorno y manejo de secretos. Integración continua básica. Cómo evaluar una aplicación con IA: conjuntos de prueba, métricas, control de costos por *token* y monitoreo.

6. Metodología

Clases de teoría los **martes** (2 horas) y sesiones de práctica los **miércoles** (2 horas). Las sesiones de teoría presentan los conceptos y el código de referencia; las de práctica son de laboratorio, con trabajo guiado sobre el mismo material.

Todo el material —*notebooks*, diapositivas y datos— vive en el repositorio del curso, y las entregas se hacen por GitHub. El uso de agentes de código está permitido y es parte del método: lo que no se acepta es entregar código que no se sabe explicar.

7. Evaluación

Se asignan **seis evaluaciones**: cinco prácticas calificadas y el trabajo final de *startup*. **Se elimina la nota más baja entre las cinco prácticas**, de modo que promedian cuatro. Cada estudiante dispone de una semana para entregar cada práctica. Los rubros usan la nomenclatura del sistema de notas de la Universidad, de modo que lo que se lee aquí es exactamente lo que aparece en el registro.

Rubro	Qué comprende	Peso (%)
Promedio prácticas	Cinco prácticas calificadas; se elimina la más baja y promedian las cuatro restantes	60
Trabajo final	Proyecto de <i>startup</i> desplegada, con <i>pitch</i> y sustentación	30
Asistencia	Asistencia y participación en las sesiones	10
Total		100

Cada práctica que cuenta aporta 15 puntos porcentuales. **Tres prácticas se rinden antes de los exámenes parciales** (1, 2 y 3) **y dos después** (4 y 5); el trabajo final de *startup* es la última evaluación del semestre.

Trabajo final: construye tu startup (30 %)

El entregable final no es un informe ni un *notebook*: es una **empresa funcional en pequeño**. Modalidad **individual** —un *solo founder* por proyecto, sin equipos—, sobre un problema real y verificable, preferentemente en Perú o América Latina.

- **Flujo de datos completo y funcionando**, de extremo a extremo: ingesta, procesamiento, modelo e interfaz.
- **Demo desplegado** en una URL pública, accesible sin clonar ni configurar nada.
- **Repositorio público en GitHub** con estructura limpia, *README* útil, *backend* inspeccionable, integración continua básica, *commits* distribuidos en el tiempo y sin credenciales filtradas.

- **Al menos dos herramientas** trabajadas en el curso (MCP, RAG, *fine-tuning*, OCR, ASR, agentes, geoespacial), con justificación de por qué esa herramienta.
- **Pitch** de 7 minutos más 3 de preguntas, con demo en vivo.

Rúbrica (escala vigesimal). Problema y validación 3; solución e *insight* 2; mercado y modelo de negocio 3; prototipo desplegado 5; repositorio 3; uso de herramientas del curso 2; *pitch* y sustentación 2. La sustentación puede mover la nota hasta ± 2 puntos.

Honestidad con la IA. Se puede y se debe usar Claude Code, Codex u otros agentes. Lo que no se acepta es no entender el código entregado: en la sustentación se pedirá explicar y modificar partes del repositorio en vivo.

8. Bibliografía

- Békés, G., & Kézdi, G. (2021). *Data Analysis for Business, Economics, and Policy*. Cambridge University Press.
- Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/en/v2>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2023). *An Introduction to Statistical Learning: With Applications in Python*. Springer.
- McKinney, W. (2022). *Python for Data Analysis* (3.^a ed.). O'Reilly. <https://wesmckinney.com/book/>
- Anthropic. *Model Context Protocol: documentación*. <https://modelcontextprotocol.io>
- LangChain. *Conceptual Guide*. <https://python.langchain.com/docs/concepts/>
- Hugging Face. *NLP Course*. <https://huggingface.co/learn/nlp-course>
- Streamlit. *Documentation*. <https://docs.streamlit.io>

9. Cronograma — Semestre 2026-II

Teoría los martes y práctica los miércoles, 7:30–9:20 am. Inicio de clases: **lunes 10 de agosto de 2026**; último día de clases: **sábado 21 de noviembre**. Durante la semana de exámenes parciales (28 set–3 oct) la Universidad suspende las clases. **Ninguna sesión del curso coincide con un feriado.**

Sem.	Fechas	Tema	Entrega
1	11–12 ago	Ciencia de datos, Git y GitHub, y agentes de código	
2	18–19 ago	Fundamentos de Python: sintaxis, funciones y clases	
3	25–26 ago	Web Scraping y APIs: Selenium, requests y consumo de servicios	Práctica 1 (mié 26 ago)

Sem.	Fechas	Tema	Entrega
4	1–2 set	MCPs en Claude Code: GitHub, Gmail y WhatsApp	
5	8–9 set	Visualización y <i>dashboards</i> : seaborn y Streamlit	Práctica 2 (mié 9 set)
6	15–16 set	Análisis geoespacial: mapas estáticos y dinámicos	
7	22–23 set	Datos raster e imágenes satelitales	Práctica 3 (mié 23 set)
—	28 set – 3 oct	<i>Semana de exámenes parciales — sin clases</i>	
8	6–7 oct	LLMs: instrucciones, salidas estructuradas y RAG	
9	13–14 oct	Hugging Face y <i>fine-tuning</i> de modelos abiertos	Práctica 4 (mié 14 oct)
10	20–21 oct	Document AI y voz: OCR, extracción estructurada y ASR	
11	27–28 oct	Agentes: crewAI, subagentes, <i>skills</i> y OpenClaw	Práctica 5 (mié 28 oct)
12	3–4 nov	Despliegue y evaluación de productos de datos	<i>Arranca el proyecto</i>
13	10–11 nov	Taller de proyecto: acompañamiento en clase	
14	17–18 nov	Presentaciones de <i>startups</i>	<i>Pitch</i> y demo
—	23–29 nov	Cierre del semestre	Entrega final

Fechas oficiales del calendario académico 2026-II

- **Matrícula extemporánea y modificación de matrícula:** sábado 8 de agosto.
- **Último día de retiro sin pago de derechos pendientes:** sábado 22 de agosto.
- **Exámenes parciales** (no hay clases esa semana): 28 de setiembre al 3 de octubre.
- **Encuestas de evaluación docente:** 2 al 15 de noviembre.
- **Último día de retiro con pago de derechos pendientes y último día de clases:** sábado 21 de noviembre.
- **Exámenes finales** (entrega del proyecto final): 23 al 29 de noviembre.
- **Devolución de calificaciones finales:** viernes 4 de diciembre, hasta las 12:00 horas.

Fuente: Calendario Académico Regular 2026, actualizado en enero de 2026.